INSTITUTO FEDERAL DE SÃO PAULO

Campus São Paulo

INTEGRANTES: [PREENCHER NOMES COMPLETOS]

TURMA: [PREENCHER]

SABOR & CLIC

Sistema de gestão para dark kitchens e espaços gastronômicos

São Paulo

2026

SUMÁRIO

1 INTRODUÇÃO E OBJETIVO

2 ATORES E REQUISITOS

3 ARQUITETURA E ORGANIZAÇÃO

4 FLUXOS E REGRAS DE NEGÓCIO

5 API E FEEDBACK

6 BANCO DE DADOS

7 SEGURANÇA E LIMITES

8 DESIGN E ACESSIBILIDADE

9 PLANO DE TESTES E DEMONSTRAÇÃO

10 REFLEXÕES SOBRE AS DECISÕES

11 ENTREGAS E PENDÊNCIAS DO GRUPO

12 REFERÊNCIAS E MATERIAIS DO PROJETO

1 INTRODUÇÃO E OBJETIVO

A Sabor & Clic é uma proposta de sistema para dark kitchens e espaços gastronômicos. O projeto integra catálogo de pratos, carrinho, pedidos, produção e reserva de bancadas. Esta documentação acompanha a complementação do projeto e deve ser revisada pelos integrantes antes da entrega acadêmica.

O sistema utiliza HTML5 e CSS3 puros, JavaScript ES6 com orientação a objetos, comunicação JSON por Fetch API, Flask e SQLite. A identidade visual mantém vermelho, cinza e branco. A interface e a API pertencem à mesma aplicação. Node.js serve aos testes do frontend, sem constituir um segundo backend de produção.

2 ATORES E REQUISITOS

O administrador gerencia pratos, preços, descrições, ficha técnica, disponibilidade e estoque. Também cadastra funcionários, bancadas e consulta relatórios. O cliente consulta o menu, organiza o carrinho, informa o endereço, confirma pedidos, acompanha o status e reserva uma bancada. O cozinheiro consulta a fila de produção, registra uma estimativa e avança o status do pedido.

Os dados do cliente incluem nome, email e endereço. O funcionário possui perfil de acesso, turno e especialidade. O histórico pertence ao cliente identificado na sessão. Ocultar controles na interface melhora a experiência, mas a autorização efetiva ocorre no Flask.

3 ARQUITETURA E ORGANIZAÇÃO

A organização segue MVC: models centralizam persistência e transações, controllers validam requisições e coordenam regras, templates e arquivos estáticos apresentam a interface. app/models contém a camada de dados, app/controllers contém rotas e app/templates contém páginas. Os scripts JavaScript separam entidades, transporte e eventos de tela.

O navegador captura um evento e valida a entrada. Uma classe organiza o estado em memória. A API JavaScript envia JSON com fetch e async/await. O Flask verifica sessão, perfil, CSRF e parâmetros. O modelo consulta o SQLite usando parâmetros vinculados e devolve dados serializáveis. A resposta JSON permite atualizar o DOM sem recarregar a página.

Usuario é a base para Cliente, Cozinheiro e Administrador. Prato, ItemPedido, Carrinho, Pedido e Reserva representam entidades da interface. Campos privados com # protegem invariantes internas. Métodos estáticos expressam operações que não dependem de uma

instância. Encapsulamento no cliente auxilia organização e não substitui validação do servidor.

4 FLUXOS E REGRAS DE NEGÓCIO

Catálogo: o administrador cria, altera ou remove logicamente um prato. A versão do registro detecta edição concorrente. O catálogo público exibe pratos disponíveis. A exclusão lógica preserva referências e o nome copiado no item mantém o histórico compreensível.

Pedido: o cliente envia identificadores e quantidades. O servidor consulta preço, disponibilidade, versão e estoque atuais dentro de uma transação. O total é calculado no servidor. O estoque representa porções de pratos, pois a especificação não fornece uma composição por insumos. A trigger faz a baixa e impede estoque negativo. Pratos legados começam sem estoque disponível até reposição explícita.

A chave opcional de idempotência evita duplicar uma compra quando a mesma requisição é repetida. A mesma chave com conteúdo diferente gera conflito. O fluxo de produção é Aguardando, Iniciando Preparo e Pronto para Entrega. A estimativa registra minutos e previsão, sem representar garantia de prazo.

Reservas: o cliente escolhe uma bancada ativa e envia início e fim com fuso horário. O servidor converte para UTC. O início deve ser futuro e estar no horizonte de 90 dias, com duração máxima de 12 horas. Dois intervalos confirmados conflitam quando inicio_novo é menor que fim_existente e fim_novo é maior que inicio_existente. Horários consecutivos são permitidos. O cancelamento exige reserva futura confirmada e autorização sobre o registro.

5 API E FEEDBACK

As respostas usam sucesso e dados ou mensagem. As falhas usam sucesso falso, mensagem e erro. O cliente trata carregamento, sucesso e erro em try/catch e apresenta mensagens visuais. As mutações enviam X-CSRFToken e o token é renovado após login e logout.

Rotas principais: /api/login, /api/cadastro, /api/logout e /api/perfil cuidam da identidade. /api/cardapio oferece GET público e POST, PUT e DELETE administrativos. /api/pedidos recebe compras, devolve o histórico permitido e permite ao funcionário atualizar status e estimativa. /api/bancadas, /api/reservas e /api/relatorios atendem locação e gestão.

Códigos HTTP distinguem autenticação, proibição, entrada inválida, recurso inexistente e conflito. A interface deve informar um conflito de versão, disponibilidade ou reserva e

solicitar revisão dos dados atuais, sem esconder a falha nem repetir a compra indefinidamente.

6 BANCO DE DADOS

As entidades centrais são clientes, funcionarios, pratos, pedidos e itens_pedido. bancadas e reservas representam locação. log_pedidos registra mudanças de status. auth_sessions e login_attempts apoiam autenticação e limitação de tentativas. O dicionário de dados descreve os campos físicos e o DER de referência mostra relações para comparação com o arquivo nativo DER_SABOR_CLIC.brM3, que requer revisão do grupo no brModelo.

O conjunto SQL segue os nomes do enunciado: 01_modelo_fisico.sql, 02_insercoes_basicas.sql, 11_queries_relatorios.sql e 20_triggers.sql. As consultas contemplam vendas, estoque e ocupação. As alterações no banco existente são aditivas. Antes de migrar um banco real, o grupo deve manter uma cópia de segurança e validar os dados preservados.

Valores monetários usam a estrutura legada do SQLite. O modelo deve validar limites e recalcular totais. Uma evolução para produção pode adotar centavos inteiros ou decimal de precisão definida. Datas de reservas usam UTC com fuso explícito. A configuração por conexão ativa chaves estrangeiras.

7 SEGURANÇA E LIMITES

As medidas a demonstrar incluem hash de senha, sessões com expiração e revogação, cookies protegidos conforme o ambiente, autorização por perfil e propriedade, CSRF nas mutações e consultas parametrizadas. Dados externos devem entrar no DOM por textContent ou criação de elementos, evitando interpolação em innerHTML.

A aplicação valida email, texto, números, datas, quantidades e transições de status. Regras importantes também residem na transação e em triggers para resistir a concorrência ou escrita direta. Mensagens de erro não devem expor SQL, senha, token ou rastreamento interno. Não existe senha administrativa pública no material de instalação.

O mapeamento ao OWASP Top 10 é uma referência para revisão, não uma certificação. Controle de acesso, injeção, autenticação, configuração, integridade e registro de eventos merecem revisão contínua. HTTPS, segredos próprios, backup, atualização de dependências e avaliação de infraestrutura são necessários antes de um uso público real.

8 DESIGN E ACESSIBILIDADE

A interface conserva vermelho #d32f2f, cinza #333333, fundo #f8f9fa e branco #ffffff. Usa tipografia Arial com alternativa sem serifa, cartões simples e navegação inferior. O guia de estilos descreve cores, estados, componentes e verificação de contraste.

O HTML deve usar cabeçalhos, seções, formulários e botões semânticos. Campos precisam de rótulos, mensagens compreensíveis e foco visível. Status devem incluir texto. O grupo deve testar teclado, zoom e larguras de celular e desktop. O Figma original não foi fornecido, portanto a correspondência com um protótipo aprovado continua pendente de comparação.

9 PLANO DE TESTES E DEMONSTRAÇÃO

Os testes automatizados devem cobrir autenticação, CSRF, permissões, manipulação de identificadores, validações, estoque insuficiente, edição concorrente, transições de status, idempotência e choque de reservas. Os testes de migração devem confirmar preservação de registros e criação dos novos campos. Os testes do DOM e de navegador devem verificar carrinho, feedback e atualização sem recarga.

A demonstração sugerida começa com catálogo e compra do cliente. Em seguida o cozinheiro define a estimativa e altera o status. O administrador repõe estoque, altera um prato e consulta relatórios. Por fim, dois clientes tentam reservar o mesmo horário e o grupo mostra o conflito. Resultados e contagens de testes devem ser consultados no registro VERIFICACAO.md atualizado após a execução, sem presumir aprovação antecipada.

10 REFLEXÕES SOBRE AS DECISÕES

1. A orientação a objetos agrupa dados e operações, reduz duplicação e permite evoluir o carrinho sem espalhar regras pelos eventos do DOM. A separação entre entidades e transporte facilita testar o estado. Herança deve expressar diferenças reais entre perfis para não criar dependências desnecessárias.

2. JSON permite que interface e backend evoluam mediante um contrato explícito. O navegador cuida da interação e o servidor decide regras e persistência. Latência pode deixar informações desatualizadas ou provocar reenvio. Carregamento visível, tratamento de erro, atualização periódica e idempotência reduzem esses efeitos.

3. Centralizar o banco no Flask evita expor credenciais e concentra autorização e transações. Consultas parametrizadas reduzem o risco de injeção. Isso também concentra carga no servidor. Transações curtas, índices e consultas limitadas ajudam desempenho.

SQLite permite apenas um escritor por vez e exige avaliar concorrência antes de ampliar o uso.

4. Se um prato for removido enquanto o cliente compra, a transação deve revalidar o estado atual e rejeitar o pedido incompatível. A versão detecta alterações conhecidas e a verificação no banco continua obrigatória. A interface explica o conflito, atualiza o menu e permite ao cliente revisar o carrinho. Não se deve aceitar silenciosamente preço ou disponibilidade antigos.

11 ENTREGAS E PENDÊNCIAS DO GRUPO

O pacote reúne código, scripts SQL e materiais acadêmicos de apoio. Permanecem sob responsabilidade do grupo: preencher nomes e turma, revisar todo o conteúdo, conferir as instruções dos professores, fornecer ou comparar o Figma original, abrir e revisar o arquivo nativo DER_SABOR_CLIC.brM3 no brModelo.

O grupo deve registrar contribuições reais no histórico Git e entregar a apresentação PDF, a documentação PDF e o projeto ZIP ou repositório no Moodle. Este material não fabrica commits nem realiza a submissão. A formatação desta minuta usa A4, fonte serifada de 12 pontos (compatível com Times), espaçamento 1,5 e margens de 3 cm superior/esquerda e 2 cm inferior/direita. A revisão acadêmica final deve conferir a norma ABNT adotada pela instituição e os critérios da disciplina.

12 REFERÊNCIAS E MATERIAIS DO PROJETO

INSTITUTO FEDERAL DE SÃO PAULO. Projeto Integrador: Sabor & Clic. Enunciado fornecido ao grupo, atualizado em 31 ago. 2026. Campus São Paulo.

SABOR & CLIC. Código-fonte, contratos de integração e scripts SQL desta entrega. Materiais locais do projeto, 2026. Consultar também GUIA_ESTILOS.md, MATRIZ_REQUISITOS.md, DICIONARIO_DADOS.docx e DER_REFERENCIA.pdf.